



Can you revise and in our ecosystem Agents are called Boomer_Angs. Picker_Ang is an original Ang. And that's the format we use to name them. Suffix is Ang or Angs. Prefix is a name related to their role, action or purpose. The prefix and suffix are attached by an underscore. So you get Picker_Ang because it is the tell selector agent.

However, there are instances where an agent doesn't have that formatted name. Like Buildsmith. Because it's in the role of smithing. So the logical name of that superhero role is to be unique. All Boomer_Angs aren't as unique as Buildsmith.

What I'm telling you is vital information. You must formulate the foundation of the roles in such a way that it's established how we develop or agents.

We also will build bots called Lil_Hawks. They are Chicken Hawks minions. The same governance will play at hand but their names are as follows. Prefix is Lil. Followed by an underscore. Here is the name that's something related to role, purpose or action. Followed by an underscore and suffix -Hawk. When they are in a swarm they're to be known as a Squad and identified individually as we will give them all identities.

You don't have to create this framework because it's been created already and we just need to allude to the existing governance and hierarchy.

Now I have everything needed. Here is the revised, doctrine-aligned implementation that codifies the naming framework and updates all agent identities accordingly.

GRAMMAR Agent Naming Doctrine

Before diving into the revisions, here is the **naming governance** as you've defined it — this is the foundational rule that should live at the top of any agent development guide:^{[1] [2]}

Boomer_Ang Naming Convention

The standard format is:

```
[Role/Action/Purpose Prefix] _ Ang
```

- The **prefix** describes what the agent *does, is, or acts like* — drawn from its role, purpose, or action
- The **suffix** is always `Ang` (singular) or `Angs` (plural collective)
- Joined by an **underscore**: `Prefix_Ang`

Examples from the established ecosystem:

- `Picker_Ang` — the selector; picks the right capability provider
- `Brief_Ang` — the briefer; creates the actionable brief from NTNTN output
- `Context_Ang` — the context keeper; owns MIM pack creation and distribution
- `Capture_Ang` — the multimodal extractor; proactively mines assets before execution
- `Plan_Ang` — the planner; owns the action graph and dependency map
- `Review_Ang` — the reviewer; runs the QA/QC checklist before every release
- `Pack_Ang` — the packager; produces the delivery bundle, manifest, and handoff artifacts

Unique Role Names (no `_Ang` suffix required):

Some agents carry a name that is itself a complete identity — a *superhero role name* — where the title alone communicates the function with enough weight and uniqueness that a suffix would diminish it. These are granted on exception, not by default:

- `BuildSmith` — the smith; forges the final build from approved plans. The craft metaphor stands on its own.
- `ACHEEVY` — the orchestrator; a proper name, not a role descriptor, because it *is* the system's will.

The rule of thumb: if the name already says what it does with personality and precision, it earns its own identity. If it doesn't, it gets the `_Ang` format.

Lil_Hawk Naming Convention

These are bots — minions operating under `Chicken_Hawk`. The format is:

```
Lil _ [Role/Action/Purpose] _ Hawk
```

- Prefix is always `Lil`
- Middle segment describes the bot's role, purpose, or action
- Suffix is always `Hawk`
- All three joined by underscores: `Lil_[Name]_Hawk`

Examples:

- `Lil_Scout_Hawk` — monitors, watches, reports back
- `Lil_Fetch_Hawk` — retrieves assets, data, or content
- `Lil_Tag_Hawk` — labels, classifies, annotates

- `Lil_Watch_Hawk` — observability; tracks board state and agent status

In swarm formation, Lil_Hawks are collectively called a **Squad** and each member carries an individual identity (a unique name within the Squad, assigned at instantiation). The governance and hierarchy that applies to Boomer_Angs applies equally to Lil_Hawks — same MIM-governed context, same typed actions, same approval classes — but they operate at the minion tier under Chicken_Hawk's direction, not ACHEEVY's direct command.^[1]

Revised Agent Roster

Here is the corrected agent roster with proper names applied across all files from the previous implementation:

Core Agents (revised names)

Old Name	Correct Name	Rationale
OrchestratorAgent	ACHEEVY	Proper name; earned identity as the orchestrator
BriefAgent	Brief_Ang	Standard _Ang format; role = briefing
ContextAgent	Context_Ang	Standard _Ang format; role = context governance
MIMContextCaptureAgent	Capture_Ang	Standard _Ang format; role = proactive capture
PlannerAgent	Plan_Ang	Standard _Ang format; role = planning the action graph
PickerAng	Picker_Ang	Already correct — underscore was missing in prior code
BuildSmith	BuildSmith	Unique role identity — earns its own name
ReviewAgent	Review_Ang	Standard _Ang format; role = review + hone
PackagingAgent	Pack_Ang	Standard _Ang format; role = packaging + delivery

Domain Boomer_Angs (revised names)

Old Name	Correct Name	Rationale
ResearchAgent	Dig_Ang	Digs for source material and research briefs
VideoUnderstandingAgent	Vision_Ang	Understands and segments video
ScriptAgent	Script_Ang	Generates narrative and script documents
CodeAgent	Build_Ang	Implements code (distinct from BuildSmith's assembly role)
QAAgent	Test_Ang	Runs the test suite and QA output

Updated File Structure

All agentId strings, class names, and file names should now follow this pattern:

```
src/agents/
  base-ang.ts           ← was base-agent.ts
  acheevy.ts            ← ACHEEVY orchestrator
  brief-ang.ts          ← Brief_Ang
  context-ang.ts        ← Context_Ang
  capture-ang.ts        ← Capture_Ang (MIM Context Capture)
  plan-ang.ts           ← Plan_Ang
  picker-ang.ts         ← Picker_Ang ✓
  buildsmith.ts         ← BuildSmith ✓
  review-ang.ts         ← Review_Ang
  pack-ang.ts           ← Pack_Ang
  domain/
    dig-ang.ts          ← Dig_Ang (Research)
    vision-ang.ts       ← Vision_Ang (Video Understanding)
    script-ang.ts       ← Script_Ang
    build-ang.ts        ← Build_Ang (Code)
    test-ang.ts         ← Test_Ang (QA)
  index.ts
```

The agentId field in each class config should match: 'brief-ang', 'picker-ang', 'buildsmith', 'acheevy', etc. — consistent with the identity format throughout. [\[2\]](#) [\[1\]](#)

Governance Summary

The naming convention is not cosmetic — it is **architectural identity**. Every new Boomer_Ang added to the ecosystem must be evaluated against two questions:

1. Does the name describe the role precisely and with enough personality to stand alone? → Grant it a unique name like BuildSmith.
2. If not → apply Prefix_Ang format.

Every new Lil_Hawk bot follows Lil_[Role]_Hawk, is assigned a unique individual identity at instantiation, and operates as part of a named Squad under Chicken_Hawk's governance — not ACHEEVY's direct command chain. [\[2\]](#) [\[1\]](#)

✱✱

1. [Conduct-your-deep-dive-to-improve.md](#)
2. [grammar_operating_guidebook.md](#)